Esthevão Marttioly

Problem Set II

Question 1

The life expectancy at birth table shows that there are only eight values informed by IBGE. In order to fulfill the information across those years, I used a cubic spline interpolation.

Year	1940	1950	1960	1970	1980	1991	2000	2010
Life Expectancy	45.5	48.0	52.5	57.6	62.5	66.9	71.1	74.4

This method was chosen because it permits non-linear dynamics within decades as shown in figure 1. Using this methodology, the best guess for the life expectancy at birth for someone born in 1996 is 69.22, which is very near to the linear interpolation fit (69.23).

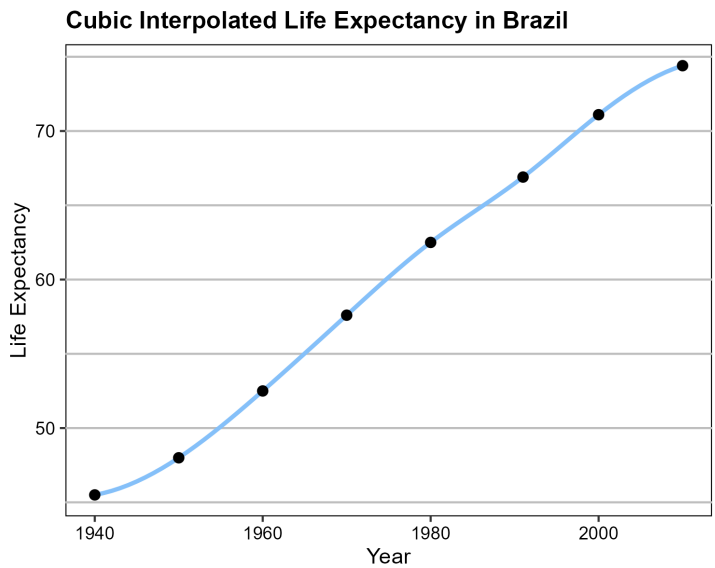

Figure 1: Cubic Interpolation for Life Expectancy Data in Brazil

Although it is smoothly and non-linear, the linear interpolation seems to be very close to it in this data, which can be a better choice thinking about computational power, mainly because this sample is small.

Question 2

(a) and (b)

We are interested in interpolating the Pareto distribution PDF for $x_m = 1$ and $\alpha = 10$:

$$f(x) = \frac{\alpha x_m^{\alpha}}{x^{\alpha+1}} = \frac{10}{x^{11}}$$

We first create a vector of trying points $x \sim \text{Unif}(1, 5)$ with sample size of 2500.

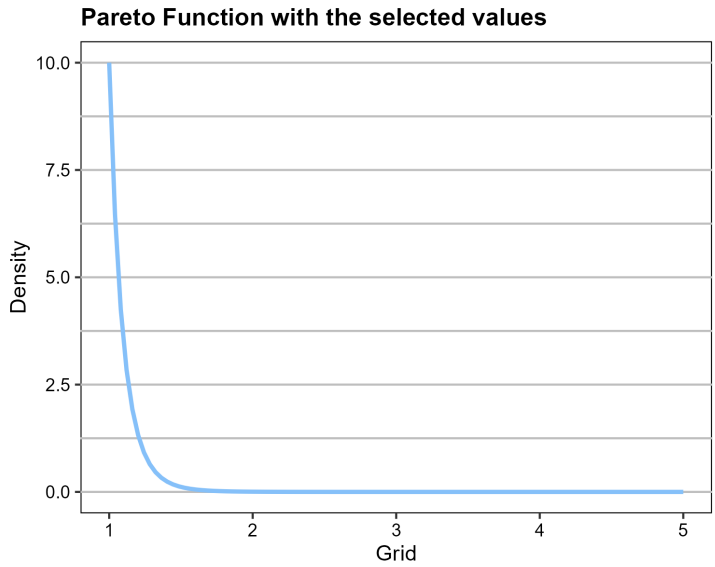

Figure 2: Pareto Density Function

(c)-(e)

Using $n \in \{10, 15, 20, 30, 50\}$ grid points t and the corresponding values $y = f(t)$, we approximate the function $f(x)$ for all 2500 evaluation points, using three interpolation methods.

For each method we compute the approximation $\hat{f}(x)$ and the interpolation error

$$e(x) = f(x) - \hat{f}(x)$$

To evaluate the performance of each method we calculate the mean squared error

$$\text{MSE} = \frac{1}{2500} \sum_{i=1}^{2500} e(x_i)^2$$

as well as the execution time required to compute the approximation for all points.

Table 1 summarizes the results for the different interpolation methods when the grid contains n points.

Table 1: Interpolation of Pareto Density Function

Grid Size	Method	MSE Equally	Time Equally	MSE Log	Time Log
10	Linear	1.103	0.0002	0.132	0.0001
10	Spline	0.410	0.0001	0.026	0.0001
10	Pchip	0.461	0.001	0.026	0.0003
15	Linear	0.386	0.0001	0.025	0.0001
15	Spline	0.096	0.0001	0.002	0.0001
15	Pchip	0.116	0.0004	0.003	0.001
20	Linear	0.164	0.0002	0.008	0.0001
20	Spline	0.027	0.0001	0.0003	0.0001
20	Pchip	0.034	0.001	0.0005	0.001
30	Linear	0.039	0.0001	0.001	0.0001
30	Spline	0.003	0.0001	0.00001	0.0001
30	Pchip	0.005	0.001	0.00004	0.001
50	Linear	0.006	0.0002	0.0002	0.0002
50	Spline	0.0001	0.0001	0.00000	0.0001
50	Pchip	0.0003	0.001	0.00000	0.001

(f)

From the results we observe that spline and PCHIP interpolation produce substantially smaller approximation errors than linear interpolation. This occurs because higher–order interpolation methods capture the curvature of the function more accurately, whereas linear interpolation only connects grid points with straight segments.

The results show that increasing the number of grid points significantly reduces the interpolation error for all methods. As the grid becomes denser, the interpolating function is constructed using more information about the underlying function, which improves the approximation.

Although spline interpolation consistently provides the lowest error, the differences between spline and PCHIP become negligible when the grid is sufficiently dense.

A log grid allocates more points near the lower bound of the domain, where the Pareto density changes more rapidly. As a result, the approximation improves significantly relative to the equally spaced grid.

Therefore, concentrating grid points in regions where the function has higher curvature leads to more accurate approximations.

Question 3

We repeat the interpolation exercise from Question 2 for the function

$$g(x) = [1 + \exp(-\eta \log(x/\xi))]^{-1} = \frac{1}{1 + x/(500e^5)}$$

with parameters $\eta = 5$, $\xi = 500$, and $x \in [200, 1000]$.

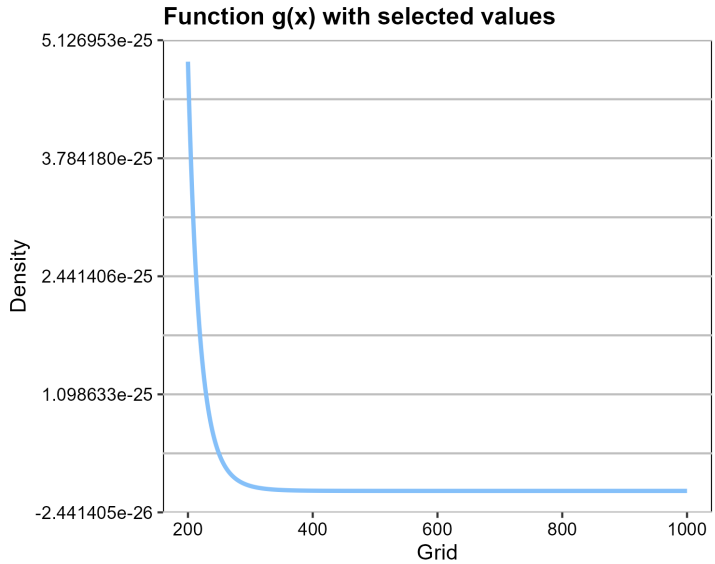

Figure 3: Function $g(x)$

The results for this function differ from those obtained in Question 2. In the previous exercise the Pareto density changes very rapidly near the lower bound of the domain, which made a log–spaced grid particularly effective. In contrast, the function $g(x)$ is smooth and monotonic over the interval $[200, 1000]$.

As shown in Table 2, the interpolation errors are extremely small for all methods and grid sizes. In most cases the mean squared error is effectively zero. This occurs because the function varies smoothly across the domain and can therefore be approximated very accurately even with relatively few grid points.

Consequently, the differences between interpolation methods become negligible. Linear interpolation, spline interpolation, and PCHIP all produce almost identical approximations of the function.

Given the shape of the function, the best grid is an equally spaced grid. The function $g(x)$ does not exhibit strong curvature near the boundaries of the domain. Therefore there is no clear benefit from concentrating grid points near the lower bound using a log–spaced grid.

Table 2: Interpolation of $g(x)$ Function

Grid Size	Method	MSE Equally	Time Equally	MSE Log	Time Log
10	Linear	0.00003	0.0001	0.00003	0.0001
10	Spline	0.00000	0.0001	0.00000	0.0001
10	Pchip	0.00000	0.0003	0.00000	0.0003
15	Linear	0.00000	0.0001	0.00001	0.0001
15	Spline	0	0.0001	0	0.0001
15	Pchip	0.00000	0.0004	0.00000	0.0004
20	Linear	0.00000	0.0001	0.00000	0.0001
20	Spline	0	0.0001	0	0.0001
20	Pchip	0	0.0005	0	0.0005
30	Linear	0.00000	0.0001	0.00000	0.0001
30	Spline	0	0.0001	0	0.0001
30	Pchip	0	0.001	0	0.001
50	Linear	0.00000	0.0002	0.00000	0.0002
50	Spline	0	0.0001	0	0.0001
50	Pchip	0	0.001	0	0.001

In summary, for smooth functions such as $g(x)$, the choice of interpolation method and grid spacing is less critical, and simple equally spaced grids combined with basic interpolation methods already provide highly accurate approximations.

Question 4

(a)

We want to find the root of the following function:

$$f(x) = x^3 + 2x + 5$$

To bracket the root we evaluate the function at two nearby points:

$$f(-2) = -7 \qquad \text{and} \qquad f(-1) = 2$$

Since the function changes sign in this interval, the Intermediate Value Theorem (IVT) implies that a root exists in $[-2, -1]$.

(b)-(d)

Using the bisection method, the root after n iterations is approximately

$$x^* \approx -1.328269 \qquad \text{with} \qquad n = 27$$

(c)

The secant method converges faster than the bisection method because it uses linear approximations of the function rather than repeatedly halving the interval. As a result, the secant method typically requires fewer iterations to reach the same tolerance.

$$x^* \approx -1.328269 \qquad \text{with} \qquad n = 7$$

(d)

Since the function is a polynomial, its derivative can be computed easily:

$$f'(x) = 3x^2 + 2$$

Using the Newton–Raphson method,

$$x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)}$$

This method exhibits quadratic convergence near the root, meaning the number of correct digits roughly doubles at each iteration. As a result, Newton's method converges much faster than both the bisection and secant methods for this problem.

$$x^* \approx -1.328269 \qquad \text{with} \qquad n = 4$$

In summary, the relative performance of the methods is

$$\text{Newton-Raphson} > \text{Secant} > \text{Bisection}.$$

Table 3: Comparing three computational methods

Method	Root	Iterations	Time (sec)
Bisection	-1.328	25	0.00003
Secant	-1.328	7	0.00002
Newton	-1.328	4	0.002